

Reviewer Pack — 2-Adic Cokernel Type of Sign Matrix Caps Randomized CC

Entry 7227efbfb367 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 01:51:43 UTC

2-Adic Cokernel Type of Sign Matrix Caps Randomized CC

Entry ID: 7227efbfb367 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 01:51:43 UTC

1. Conjecture

Field A (mathematical branch): Hall algebra of finite abelian 2-groups / Cohen-Lenstra heuristics on integer-matrix cokernels — the partition $\lambda_2(M) \vdash v_2(|\det M|)$ encoding the 2-Sylow structure of $\text{cok}(M) = \mathbb{Z}^n / M \cdot \mathbb{Z}^n$ as a finite abelian 2-group (Hall 1959; Macdonald 1995 ‘Symmetric Functions and Hall Polynomials’ Ch. II ‘The Hall algebra’; Cohen-Lenstra 1984 ‘Heuristics on class groups’; Friedman-Washington 1989 random matrix cokernels; Wood 2017 ‘Random integral matrices’). The invariant is computed via Smith normal form mod 2^N in $O(n^3 \cdot \log n)$ pure-Python big-int Gaussian elimination, sidestepping any Coxeter/Hecke machinery. An arXiv search ‘Cohen-Lenstra’ AND (‘communication complexity’ OR ‘sign rank’ OR ‘discrepancy’) returns 0 direct hits with <5 adjacent papers (all on lattice-based cryptography or chip-firing critical groups, never on CC matrices). Distinct from blacklisted 2-Sylow rank of Tseitlin critical group $K(G)$ (an invariant of a charged GRAPH, not a sign matrix) and from Specht-isotype / cycle-type / RSK / hook-length attempts (which projected via S_n -characters or partition statistics, never via the Hall-algebra 2-adic cokernel type of an integer matrix). **Field B** (complexity object): Two-party randomized communication complexity $R_{\{1/3\}}(M)$ for $\{\pm 1\}$ -sign matrices $M \in \{\pm 1\}^{\{n \times n\}}$, in the Yao 1979 / Razborov 1992 DISJ regime, accessed via the Linial-Shraibman γ_2 / Klartag-Regev corruption lower bound and the Forster sign-rank route to UPP^{cc} . The invariant $\lambda_2(M)$ is STRUCTURAL on the sign assignment: it is preserved under the full CC symmetry group (row/column permutation + signed-diagonal pre/post conjugation $D_1 \cdot M \cdot D_2$ with $D_i \in \{\pm 1\}^n$ diagonal), since these act as $\mathbb{Z}$ -module automorphisms of $\mathbb{Z}^n$ and leave the cokernel isomorphism class fixed — but it is NOT a poly-time-computable property of the underlying Boolean function (two CC matrices with identical function up to relabeling can have differing λ_2 after lift to $\mathbb{Z}$), shielding from Razborov-Rudich.

Statement:

For every sign matrix $M \in \{\pm 1\}^{\{n \times n\}}$, let $v_2(|\det M|)$ be the 2-adic valuation of the determinant computed over $\mathbb{Z}$, and define $\rho(M) := v_2(|\det M|) / n \in [0, (\log_2 n)/2]$. We conjecture there exists an absolute constant $c \in (0, 4)$ such that for all $n \geq 4$ and all $M \in \{\pm 1\}^{\{n \times n\}}$ with $\gamma_2(M) \leq g$ (the Linial-Shraibman γ_2 -norm, equivalent to $R_{\{1/3\}}(M)$ up to log factors): $\rho(M) \leq c \cdot (\log_2 g) / (\log_2 n)$. One sign matrix M with $\rho(M) \cdot \log_2 n > c \cdot \log_2 \gamma_2(M)$ for $c = 4$ falsifies the conjecture.

Rationale (proposer’s reasoning):

Sign matrices with arithmetic structure that forces $|\det M|$ to absorb a large 2-power (Sylvester-Hadamard, IP_k, Paley) are precisely those where the $\mathbb{Z}$ -module $\text{cok}(M)$ is far from the Cohen-Lenstra random model — and these are also the canonical hard CC functions (IP_k achieves $R = \Omega(k)$, Hadamard achieves $\gamma_2 = \Theta(\sqrt{n})$). Conversely, low-CC ma-

trices admit small monochromatic-rectangle covers, which when lifted to $\mathbb{Z}$ force $\text{cok}(M)$ to have small 2-Sylow rank because the cover provides a near-rank-1 integer decomposition. The Hall-algebra 2-Sylow type encodes finer structure than rank alone (it tracks the elementary-divisor partition), so it can detect structured matrices that fool spectral / Frobenius-norm CC bounds.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d06742b46c4174d7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 630 instances (3 families $\times$ 7 sizes $\times$ 30 seeds), compute $\Delta(M) = \rho(M) \cdot \log_2 n - 4 \cdot \log_2 g(M)$ where g is the spectral γ_2 lower-bound proxy. SUPPORTED iff $\max \Delta \leq 0$ across all instances AND mean $\Delta \leq -0.25$; FALSIFIED if any single instance yields $\Delta > 0.5$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Smith normal form sign matrix communication complexity - 2-adic valuation determinant gamma_2 norm sign rank - cokernel integer matrix discrepancy Linial Shraibman

Top relevant hits considered: - [http://arxiv.org/abs/1703.01572v1] The Smith normal form of a specialized Giambelli-type matrix - [http://arxiv.org/abs/2211.09736v2] Equidistributions of Sign Patterns of the Liouville Function and Normal Numbers - [http://arxiv.org/abs/1602.00166v2] Smith Normal Form in Combinatorics - [http://arxiv.org/abs/1211.0398v1] Extending valuations to formal completions - [http://arxiv.org/abs/1503.07648v2] Sign rank versus VC dimension - [http://arxiv.org/abs/2203.02197v1] The 2-adic valuation of the general degree 2 polynomial in 2 variables - [http://arxiv.org/abs/0809.2093v2] An approximation algorithm for approximation rank - [http://arxiv.org/abs/1706.02207v3] On The Communication Complexity of High-Dimensional Permutations

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_mod(A, m):
    n = len(A)
    result = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            result[i][j] = (A[i][j] % m + m) % m
    return result

def matrix_multiply(A, B, m):
    n = len(A)
    result = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                result[i][j] = (result[i][j] + A[i][k] * B[k][j]) % m
    return result

def matrix_power(A, k, m):
    n = len(A)
    result = [[0]*n for _ in range(n)]
    for i in range(n):
        result[i][i] = 1
    while k > 0:
        if k % 2 == 1:
            result = matrix_multiply(result, A, m)
        A = matrix_multiply(A, A, m)
        k //= 2
    return result

def linial_shraibman_gamma(M):
    n = len(M)
    M_T_M = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i, n):
            sum_val = 0
            for k in range(n):
                sum_val += (M[i][k] * M[j][k]) % n
            M_T_M[i][j] = sum_val
            M_T_M[j][i] = sum_val

eigenvalues = []
for i in range(1, 20): # Limit to 20 iterations for practical purposes
    v = [random.randint(0, n-1) for _ in range(n)]
```

```

    v_norm = sum(v[i]**2 for i in range(n)) ** 0.5
    v = [v[i] / v_norm for i in range(n)]
    for _ in range(i):
        v_next = [M_T_M[j][i] * v[j] for j in range(n)]
        v_next_norm = sum(v_next[j]**2 for j in range(n)) ** 0.5
        v_next = [v_next[i] / v_next_norm for i in range(n)]
    eigenvalues.append(sum(v_next[j]**2 for j in range(n)))

    return max(eigenvalues) / n

def is_sign_matrix(M):
    return all(x == 1 or x == -1 for row in M for x in row)

def det_mod_2k(M, k):
    n = len(M)
    A = matrix_mod(M, 2**k)
    for i in range(n):
        if A[i][i] == 0:
            for j in range(i+1, n):
                if A[j][i] != 0:
                    A[i], A[j] = A[j], A[i]
                    break
            else:
                return 0
        for j in range(n):
            if i != j:
                factor = (A[j][i] * pow(A[i][i], -1, 2**k)) % (2**k)
                for l in range(n):
                    A[j][l] = (A[j][l] - factor * A[i][l]) % (2**k)
    det = 1
    for i in range(n):
        det = (det * A[i][i]) % (2**k)
    return det

def v_2(x):
    if x == 0:
        return float('inf')
    count = 0
    while x % 2 == 0:
        x //= 2
        count += 1
    return count

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 12, 16, 20, 24, 32, 40]
    results = []

    for n in n_values:
        for _ in range(30):
            M = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
            if not is_sign_matrix(M):
                continue

            det = abs(det_mod_2k(M, n * math.ceil(math.log2(n))))
            rho = v_2(det) / n

```

```

    g = linial_shraibman_gamma(M)

    results.append({
        "n": n,
        "rho": rho,
        "g": g,
        "instance_type": "uniform"
    })

max_rho_g_diff = -float('inf')
for res in results:
    diff = res["rho"] * math.log2(res["n"]) - 4 * math.log2(res["g"])
    if diff > max_rho_g_diff:
        max_rho_g_diff = diff

return {
    "metric_name": "max_rho_g_diff",
    "metric_value": max_rho_g_diff,
    "instances_tested": len(results),
    "conjecture_holds": max_rho_g_diff <= 0,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 31)]

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")

    results = []
    for seed in seeds:
        with open(f"trial_{seed}.json", "r") as f:
            trial = json.load(f)
            results.append(trial)

    max_rho_g_diffs = [x["metric_value"] for x in results]
    support_fraction = sum(x["conjecture_holds"] for x in results) / len(results)

    if all(x["conjecture_holds"] for x in results):
        print(f"RESULT: SUPPORTED mean={sum(max_rho_g_diffs)/len(max_rho_g_diffs)} std=0.0 support")
    elif any(not x["conjecture_holds"] for x in results) and max_rho_g_diffs[-1] > 0.5:
        print(f"RESULT: FALSIFIED counterexample=\"max_rho_g_diff_exceeded\" first_failing_seed={s}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(seeds)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbb8938a.py", line 151, in <module>

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbb8938a.py", line 122, in run_trial
    det = abs(det_mod_2k(M, n * math.ceil(math.log2(n))))
    ~~~~~^~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbb8938a.py", line 94, in det_mod_2k
    factor = (A[j][i] * pow(A[i][i], -1, 2**k)) % (2**k)
    ~~~~~^~~~~~
```

6

3. The full LLM transcripts are in `research/audit/7227efbfb367.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7227efbfb367.tar.gz` (if generated)